Reliable Text-to-SQL Assistant: approach, evaluation, limitations

Stack: Python, SQLite (Chinook, 11 tables), sqlglot. **All results use gemini-flash-lite-latest** (Google Gemini free API; an alias), temperature 0. Azure OpenAI (gpt-4.1-mini) was set up first but quota errors prevented a usable evaluation, so it is not evaluated. Provider and model are set in .env.

1. Approach and reasoning

question -> schema context -> one LLM decision {answer | clarify | refuse} -> SafeDB (4 layers) -> on error: repair loop (max 2) -> SQL + table + one-line answer

Choice	What I did and why	Alternatives considered
Model	Thin wrapper returning validated JSON via a forced tool call (no text parsing); swappable providers. One cheap model evaluated because free quotas limited more.	Claude, Azure GPT, local open-weight: supported, not evaluated.
Schema context	Full schema (~1.1k tokens) read from PRAGMA, plus **exact value lists** for small columns ('Hip Hop/Rap', not 'Hip-Hop'), a short **glossary** of what DDL cannot say (revenue = UnitPrice*Quantity, dates are text, no cost/payment data) and 5 fixed few-shots, none in the test set. 11 tables fit easily, so retrieval mostly adds a way to be wrong.	**Retrieved schema** (BM25 keyword search, top-3 tables + FK-path bridge tables), measured as system C. Dynamic few-shot needs a labelled corpus.
Read-only, bounded	The prompt is not a control. (1) sqlglot AST check: one SELECT/CTE/set-operation statement, table allow-list, no write nodes; (2) connection opened mode=ro + query_only; (3) SQLite **authorizer** allow-listing operations, tables, functions; (4) timeout, size limits, 200-row cap. Tests show layers 2+3 block writes even with layer 1 bypassed.	Keyword blocklist (bypassable); read-only DB user (SQLite has none); container sandbox (more infra).
Failure handling	*Invalid SQL*: DB error fed back, 2 retries max. *Empty result*: reported honestly. *Ambiguous*: ONE clarifying question with 2-4 options. *Out of scope*, *missing data*, *writes*, *injection*: refuse; no SQL runs. A model-written write is refused, never repaired.	Separate router model: more cost, no schema knowledge.
Dates	Data ends 2013-12-22, so “last year” = 2012, stated in assumptions.	Wall-clock year returns nothing.

2. Evaluation

Test set (55, appendix): 12 easy, 15 medium, 14 hard (4-5 table joins, shares, LAG, nth-per-group, anti-joins, set logic, self-join), 6 to **clarify**, 8 to **refuse**. References were run and checked, top-N ties excluded. A perfect “oracle” must score 100% and a junk model under 10% (unit tests).

Correctness = result comparison, not SQL text. Column names/order ignored; extra columns allowed, missing fail; row order ignored unless explicitly ranked (ties may swap); row count exact; numbers within 0.011 (a lenient 1-

decimal band is reported separately); text case-sensitive; NULL equals NULL only. Clarify/refuse pass only if the system took that action and ran no SQL.

Failure types (auto-labelled by SQL structure, then checked by hand): wrong table / join / filter / aggregation / ordering-limit / output columns, rounding, API error, over-clarification, over-refusal.

Systems: A full schema, no repair; B = A + repair; C = B with retrieved schema. One run each (noise not measured); differences judged by paired exact McNemar test and paired bootstrap 95% CI.

Results (4 November 2026, one repeat per system)

System	Overall	Easy	Medium	Hard	Clarify	Refuse	Input tok/q
A full, no repair	100.0% (55/55)	100%	100%	100%	100%	100%	2,523
B full + repair	98.2% (54/55)	100%	100%	92.9%	100%	100%	2,574
C retrieved + repair	98.2% (54/55)	100%	93.3%	100%	100%	100%	1,858

Failures. B failed H08: the model also used December 2012, so January shows a change of 0.0 where the reference gives NULL (other months identical). C failed M09: all values correct, but the full name came back as two columns; the auto-label (wrong_ordering_or_limit) was wrong, showing labels are heuristic. Neither is clearly wrong; accepting both gives 100% for all, a post-hoc judgment, so strict scores are the headline.

Is the difference real? No. A to B: -1.8 points, CI [-5.5, 0.0], p = 1.0. B to C: 0.0, CI [-5.5, +5.5], p = 1.0. One-question gaps are within chance at n = 55. Only cost differs: retrieval used about 26% fewer input tokens at similar accuracy. Scores are near the ceiling, so this set cannot separate versions, and repair was needed on about one question (1.02 model calls per question), so A vs B had little chance to differ. The sandbox never triggered because the model refused writes itself; unit tests, not this run, evidence it.

A defect found by the evaluation. My first run scored A 96.4%, B 94.5%, C 94.5%. Review showed my validator rejected valid INTERSECT/EXCEPT queries as writes (H07, H10), two C failures were API errors, and M08 did not say how to label quarters. I fixed the validator with regression tests, labelled API errors separately and reworded M08 (a post-result change, so a disclosed adjustment). First-run files are kept in results_run1_before_fix.

3. Limitations

- Test set is small (54/55 is roughly 90-100%), written by me, and too easy for this model; clarify (6) and refuse (8) are a coarse signal.
- One model, one repeat: noise unmeasured; the -latest alias can change; free-tier latency not reported.
- Revenue and “last year = 2012” defaults are my convention. Failure labels are heuristic.
- The one-line answer was off during evaluation and is not evaluated; it is limited to returned rows.
- Safety tested on a fixed attack list, not fuzzed; in-process SQLite limits are not multi-tenant isolation.

4. With more time

1. A harder, larger test set (150+ questions, several authors, paraphrase groups) with repeats to measure noise.
2. Execution-guided self-consistency to attack silent errors; a faithfulness check for the answer sentence.
3. Compare Claude, Azure GPT and a local model through the same interface.
4. Container execution, rate limits, audit logging.
5. Embedding-based schema retrieval for databases with hundreds of tables (current retrieval is keyword-only).